

Transformers

Explanation

Before 2017, language models read text one word at a time, left to right, and tended to forget the start of a long sentence by the time they reached the end. The **Transformer** changed that. It looks at all the words at once and, for each word, works out which other words matter most for understanding it. That mechanism is called **attention**.

Take the sentence "The animal didn't cross the street because it was tired." To understand "it", you need to connect it to "animal", not "street". A Transformer does exactly that: when it processes "it", it pays strong attention to "animal". Because attention is calculated for every word in parallel, Transformers also train much faster on GPUs, which is what made today's huge models possible.

How a transformer processes text

1. **Tokenize** the text into pieces called tokens (whole words or parts of words)
2. **Embed** each piece as a **vector** of numbers
3. Add position information, because attention on its own has no sense of word order
4. Run it through a stack of transformer blocks, where each block is attention followed by a feed-forward network
5. **Predict** the output, usually the next token

Key terms

Term	What it means
1. Transformer	A neural network architecture based on attention that processes all tokens in parallel; the foundation of modern LLMs.
2. Attention	A mechanism that lets the model weigh how relevant every other piece of the input is when processing one piece.
3. Self-attention	Attention applied within a single sequence, so every token can look at other tokens in the same text (in decoder models, only the tokens before it).
4. Multi-head attention	Running several attention computations in parallel so the model can capture different kinds of relationships at once.
5. Encoder / Decoder	Encoders build a representation of the input (BERT); decoders generate output token by token (GPT, Claude, Llama).

Architecture diagram

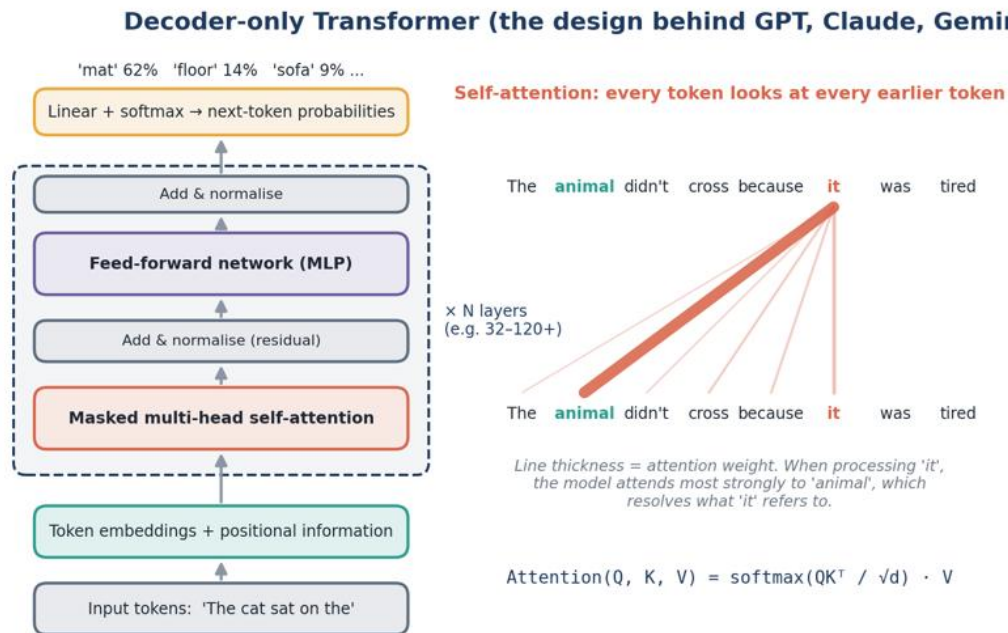


Figure 1. A decoder-only transformer (left) and the attention pattern for the word “it” (right).

How attention works: Query, Key and Value

Each token’s vector is multiplied by three learned weight matrices to produce three new vectors:

- **Query (Q):** what this token is looking for
- **Key (K):** what this token offers to others
- **Value (V):** the information this token passes along if it’s selected

In matrix form, with X holding every token’s embedding: $Q = X \cdot W_Q$ $K = X \cdot W_K$ $V = X \cdot W_V$. The three W matrices start out random and are learned during training.

Then, for every token:

1. Compare its query against every token’s key using a dot product. A high score means “relevant to me.”
2. Divide each score by $\sqrt{d_k}$, the square root of the key size. Without this, large scores push softmax to extreme 0/1 values and training becomes unstable.
3. Apply softmax so the scores become weights that sum to 1.
4. Multiply each token’s value vector by its weight and add them up. The result is the token’s new, context-aware vector.

The whole calculation as one formula:

$$\text{Attention}(Q, K, V) = \text{softmax}(QK^T / \sqrt{d_k}) \cdot V$$

Multi-head attention and masking

Multiple heads: the model runs several attention calculations side by side, each with its own W_Q , W_K and W_V . GPT-2 small, for example, has 12 heads per layer, each working on 64 of the 768 numbers in

a token's vector. Nobody assigns the heads jobs; each learns its own pattern during training (for example, looking at the previous word, or linking a pronoun to its noun). Their outputs are joined back together and mixed by one more matrix.

Causal mask: in models that generate text, a token must not see the tokens after it. Scores for future positions are set to $-\infty$ before softmax, which makes their weights zero.

Inside each transformer block

- **Feed-forward network (MLP):** a small two-layer network applied to each token on its own. Attention is where tokens share information; the feed-forward network is where each token processes what it gathered.
- **Residual connections:** each sub-layer's output is added to its input instead of replacing it, which lets very deep stacks train.
- **Layer normalization:** keeps the numbers in a stable range. Most modern models normalize before each sub-layer rather than after it as drawn in Figure 1.

Training and generating text

- **Pretraining:** the model reads huge amounts of text and learns to predict the next token. No labeling is needed, because the text supplies its own answers.
- **Post-training:** fine-tuning on example conversations and on human or AI feedback turns the raw model into an assistant that follows instructions.
- **Generation:** predict a probability for every possible next token, pick one, add it to the text and repeat. Temperature, top-k and top-p control how random the pick is.

Example

- For visualization, check out the [Transformer Explainer](#).
- For the working implementation, check out the [Transformer Attention Walkthrough notebook](#).